

HTML5 & CSS: Structured Web Pages

Lecture 2

Skills to build **well-structured, semantic, and accessible web pages** using HTML5, and to apply **basic CSS** to present content clearly and consistently.

Learning Objectives

By the end of this course, learners will be able to:

- ▶ Explain why **structure** is critical for accessibility, SEO, and maintainability
- ▶ Create a valid **HTML5 document structure** using essential tags
- ▶ Use **semantic HTML5 elements** appropriately to convey meaning
- ▶ Organize information with **lists, tables, and forms**
- ▶ Embed and manage **multimedia elements** responsibly
- ▶ Apply **basic CSS** to control layout, typography, and visual hierarchy
- ▶ Build a **simple, complete mini-page** combining HTML structure and CSS styling

Plan

1. Introduction: Why Structure Is Important on the Web
2. HTML5 Basics: Document Structure and Basic Tags
3. Semantic HTML5 Tags
4. Lists, Tables, and Forms for Organizing Data
5. Embedded Elements and Multimedia
6. CSS
7. Summary

Introduction: Why Structure Is Important on the Web

- What is **web structure**
- Difference between *content*, *structure*, and *presentation*

Why structure matters:

- Accessibility (screen readers, assistive tech)
- SEO (search engines understand pages better)
- Maintainability and scalability
- Team collaboration
- Real-world analogy: building a house (skeleton vs decor)

What Is HTML5?

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Title of the document</title>
  </head>

  <body>
    Content of the document.....
  </body>
</html>
```

HTML = HyperText Markup Language

HTML5 as the modern standard

What HTML does:

- Defines page structure
- Describes meaning of content
- What HTML does **not** do:
 - Styling (CSS)
 - Logic (JavaScript)

Basic HTML5 Document Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Web Page</title>
</head>
<body>
  Content goes here
</body>
</html>
```

► These tags should be placed underneath each other **at the top of every HTML page** that you create.

► [<!DOCTYPE html>](#) – This tag **specifies the language** you will write on the page. In this case, the language is HTML 5.

► [<html>](#) – This tag signals that from here on we are going to write in HTML code.

► [<head>](#) – This is where all the **metadata for the page** goes – stuff mostly meant for search engines and other computer programs.

► [<body>](#) – This is where the **content of the page** goes.

HTML Page Structure



```
<!DOCTYPE html>  
<html>  
<body>  
  
<h1>My First Heading</h1>  
  
<p>My first paragraph.</p>  
  
</body>  
</html>
```

Web Browser

My First Heading
My first paragraph.

Why Semantic HTML Matters

- ▶ Semantic = meaningful
- ▶ Helps browsers and search engines
- ▶ Improves accessibility
- ▶ Makes code easier to read

✗ Non-semantic:

```
<div class="header"></div>
```

✓ Semantic:

```
<header></header>
```

Main Semantic HTML5 Tags

`<header>` — page or section header

`<nav>` — navigation links

`<main>` — main content

`<section>` — thematic grouping

`<article>` — independent content

`<aside>` — side content

`<footer>` — footer information

```
<a href="http://www.google.com">This is a link to Google</a>
```

Bold Text:

```
<b>This is my Text</b>
```

Paragraph:

```
<p>My first paragraph.</p>
```

Headers:

```
<h1>This is my Header</h1>
```

```
<h2>This is my Header</h2>
```

```
<h3>This is my Header</h3>
```

Line Break:

```
This is my Text
```

```
<br>
```

```
This is also my Text
```

Title:

```
<title>This is my Title</title>
```

Comments:

```
<!-- Write your comments here -->
```

Image:

```

```

Semantic Page Layout Example

- ▶ Clear visual and logical hierarchy
- ▶ Easy to style with CSS

```
<header>
  <nav></nav>
</header>

<main>
  <section>
    <article></article>
  </section>
  <aside></aside>
</main>

<footer></footer>
```

Lists for Organizing Information

Unordered List

```
<ul>  
  <li>Item 1</li>  
  <li>Item 2</li>  
</ul>
```

Ordered List

```
<ol>  
  <li>Step 1</li>  
  <li>Step 2</li>  
</ol>
```

Description List

```
<dl>  
  <dt>HTML</dt>  
  <dd>Markup language</dd>  
</dl>
```

Tables: Structured Data

- ▶<table> – container
- ▶<tr> – row
- ▶<th> – header cell
- ▶<td> – data cell

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Alice</td>
    <td>25</td>
  </tr>
</table>
```

Forms: User Input

- ▶ `<form>` – input container
- ▶ `<input>` – user data
- ▶ `<label>` – accessibility
- ▶ `<button>` – action

```
<form>  
  <label>Email:</label>  
  <input type="email">  
  <button>Submit</button>  
</form>
```

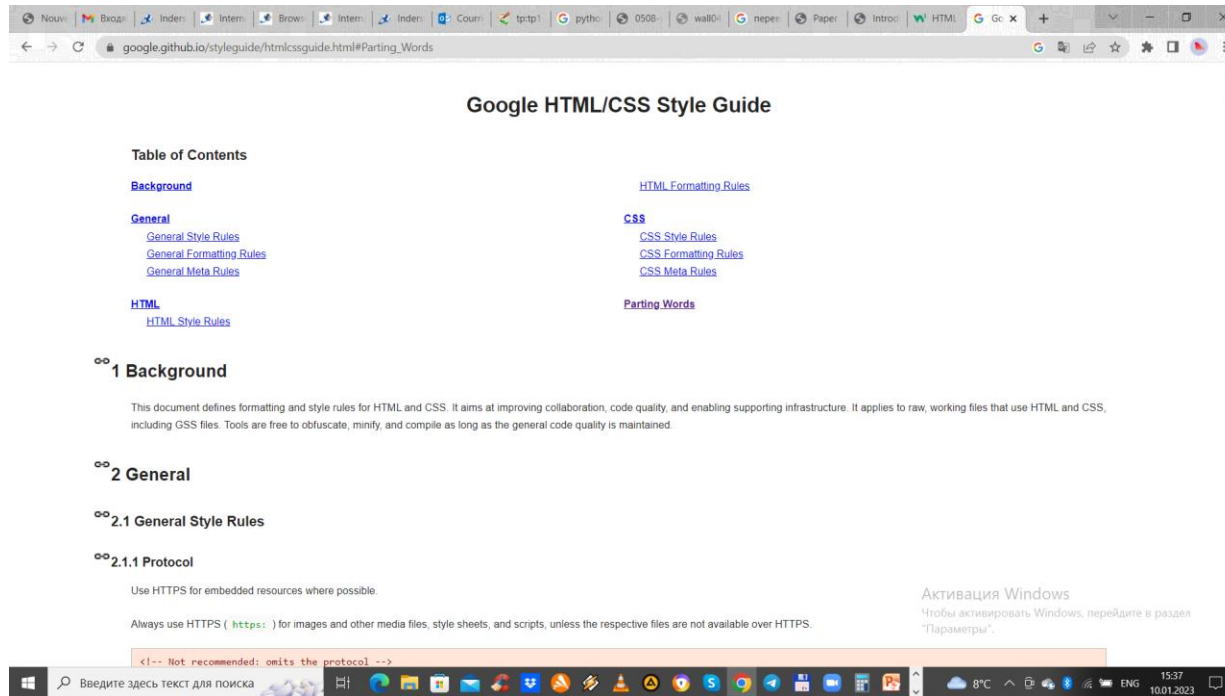
Embedded Elements

- ▶ Images: `<img>`
- ▶ Video: `<video>`
- ▶ Audio: `<audio>`
- ▶ External content: `<iframe>`

```
<video controls>  
  <source src="video.mp4">  
</video>
```

- Always use alt for images
- Optimize file size
- Use controls for audio/video
- Avoid autoplay

Multimedia Best Practices

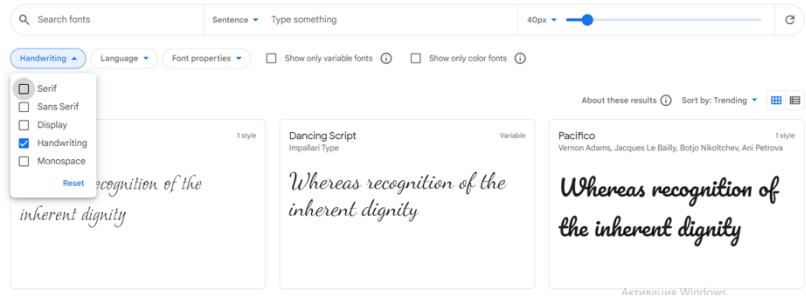


Useful link to explore

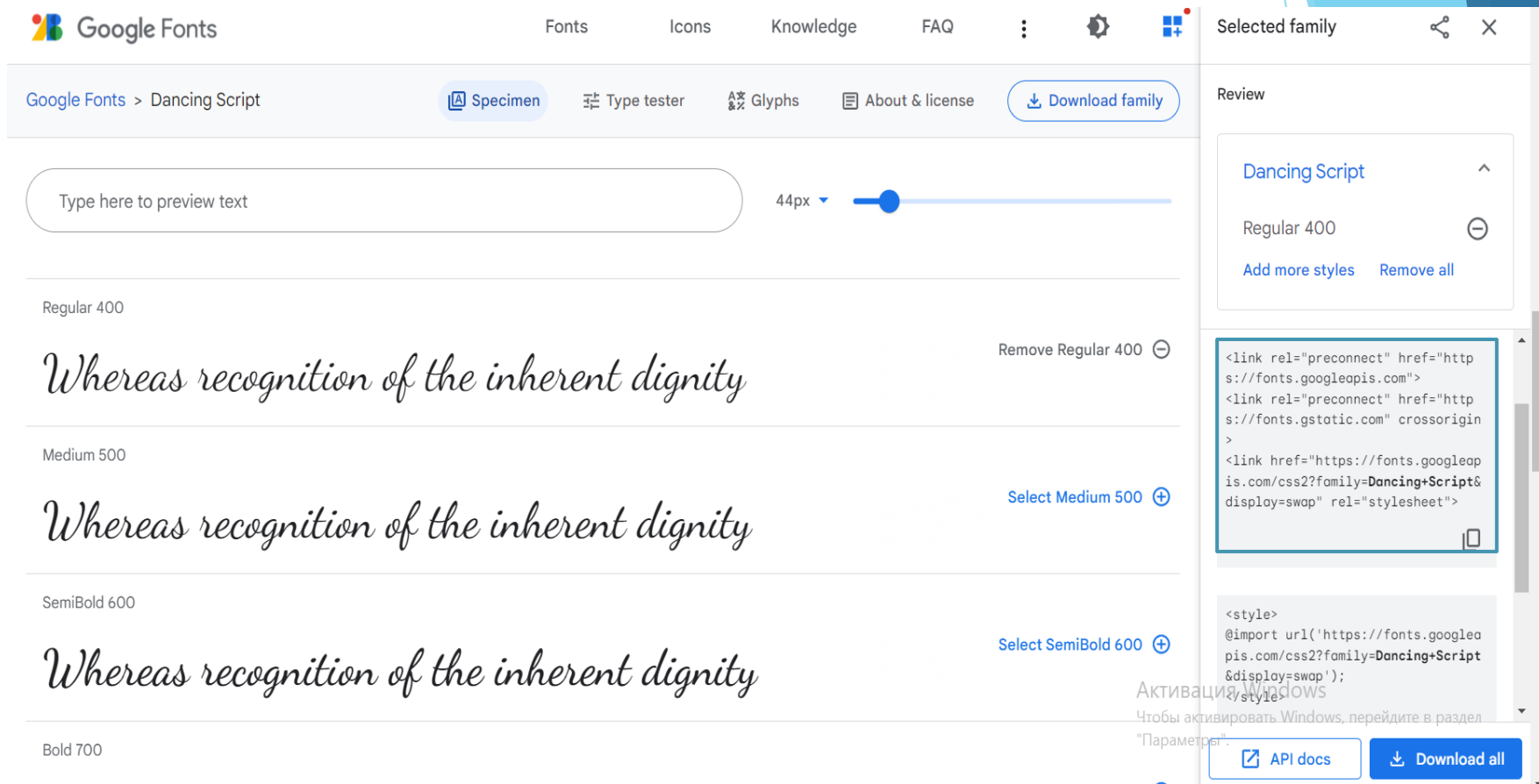
<https://google.github.io/styleguide/htmlcssguide.html>

How to Use Google Fonts in HTML and CSS

Open [Google fonts](https://fonts.google.com/) official website and the following window will appear and Select the category/ font family of your choice:



Select the google font of your choice and click on the “select this style”, Copy the link of the selected style from the window that appear on the right side of the screen and put it in HTML



Google Fonts

Fonts Icons Knowledge FAQ

Google Fonts > Dancing Script

Specimen Type tester Glyphs About & license Download family

Type here to preview text 44px

Regular 400

Whereas recognition of the inherent dignity

Remove Regular 400

Medium 500

Whereas recognition of the inherent dignity

Select Medium 500

SemiBold 600

Whereas recognition of the inherent dignity

Select SemiBold 600

Bold 700

Selected family

Review

Dancing Script

Regular 400

Add more styles Remove all

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Dancing+Script&display=swap" rel="stylesheet">
```

```
<style>
@import url('https://fonts.googleapis.com/css2?family=Dancing+Script&display=swap');
</style>
```

API docs Download all

Copy the CSS rules for font-family and paste it in the CSS file:

Google Fonts

Fonts Icons Knowledge FAQ

Google Fonts > Dancing Script

Specimen Type tester Glyphs About & license Download family

Regular 400

Whereas recognition of the inherent dignity

Remove Regular 400

Medium 500

Whereas recognition of the inherent dignity

Select Medium 500

SemiBold 600

Whereas recognition of the inherent dignity

Select SemiBold 600

Bold 700

Whereas recognition of the inherent dignity

Select Bold 700

Selected family

Review

Dancing Script

Regular 400

Add more styles Remove all

```
<style>
@import url('https://fonts.googleapis.com/css2?family=Dancing+Script&display=swap');
</style>
```

CSS rules to specify families

```
font-family: 'Dancing Script', cursive;
```

Introduction to CSS

CSS = Cascading Style Sheets

Controls:

- Colors
- Fonts
- Layout
- Spacing

Separation of concerns:

- HTML — structure
- CSS — appearance

CSS Syntax Basics

CSS (Cascading Style Sheets) is the language we use to style an HTML document.

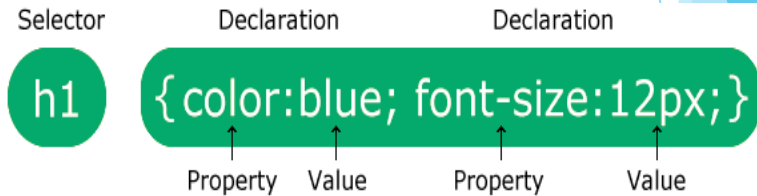
CSS describes how HTML elements should be displayed.

Used to control the style and layout of multiple Web pages all at once

```
selector {  
  property: value;  
}
```

Example:

```
p {  
  color: black;  
  font-size: 16px;  
}
```



Connecting CSS to HTML

Ways to add CSS:

- ▶ External (recommended)
- ▶ Internal
- ▶ Inline (not recommended)

```
<link rel="stylesheet" href="style.css">
```

Inline - by using the style attribute inside HTML elements.

- An inline style loses many of the advantages of a style sheet (by mixing content with presentation). Use this method sparingly.

Internal - by using a <style> element in the <head> section.

- An internal style sheet should be used when single document has a unique style
- You define it in the head section of an HTML page, inside the <style>tag

External - by using a <link> element to link to an external CSS file.

- is ideal when the style is applied to many pages. With it you can change the look of an entire Web site by changing just one file.
- Can be written in any text editor. The file should not contain any html tags.
- Must be saved with .css extension

Internal Style Sheet Example

You define internal styles in the head section of an HTML page, inside the <style> tag, like this:

```
<head>
<style>
body {
    background-color: linen;
}
h1 {
    color: maroon;
    margin-left: 40px;
}
</style>
</head>
```



Students: Try this Example

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
    background-color: linen;
}
h1 {
    color: maroon;
    margin-left: 40px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

External Style Sheet Example

Each HTML page must include a link to the style sheet with the <link> tag. The <link> tag goes inside the head section:

```
<head>  
<link rel="stylesheet" type="text/css" href="mystyle.css">  
</head>
```

An example of a style sheet file called "myStyle.css", is shown below:

```
body {  
    background-color: lightblue;  
}  
  
h1 {  
    color: navy;  
    margin-left: 20px;  
}
```

CSS Properties

Text Color

```
body {  
    color: blue;  
}  
  
h1 {  
    color: #00ff00;  
}  
  
h2 {  
    color: rgb(255,0,0);  
}
```

Text Alignment

```
h1 {  
    text-align: center;  
}  
  
p.date {  
    text-align: right;  
}  
  
p.main {  
    text-align: justify;  
}
```

Background Color

```
body {  
    background-color: lightblue;  
}
```

Text Size

```
h1 {  
    font-size: 40px;  
}  
  
h2 {  
    font-size: 30px;  
}  
  
p {  
    font-size: 14px;  
}
```

Text Font

```
p {  
    font-family: "Times New Roman", Times, serif;  
}
```



Students: Create a Style Sheet (.CSS) and a HTML page where you use these Properties

- Overusing <div> instead of semantic tags
- Forgetting alt attributes
- Mixing HTML and CSS logic
- Poor indentation

Common Beginner Mistakes

- HTML provides structure
- Semantic tags improve clarity
- Lists, tables, and forms organize data
- CSS styles the structure

Summary

Questions

1. What does HTML stand for?
 - A. Hyper Text Markup Language
 - B. Hot Mail
 - C. How to Make Lasagna
2. How many tags are in a regular element?
 - A. 2
 - B. 1
 - C. 3
3. What is the difference between an opening tag and a closing tag?
 - A. Opening tag has a / in front
 - B. Closing tag has a / in front
 - C. There is no difference

4. `< br / >` What type of tag is this?

- A. Break tag
- B. A broken one
- C. An opening tag

5. `< body >` Is this an opening tag or a closing tag?

- A. Opening
- B. Closing

6. `< / body >` Is this an opening tag or a closing tag?

- A. Opening
- B. Closing

7. Where is the meta tag only found?

- A. The last page
- B. The home page
- C. The second page

8. Which is the correct way to tag an image?

- A. `src="image.jpg/gif" alt="type some text"`
- B. `Src="image.jpg/gif" alt="type some text"`
- C. none is correct

9. What is an element that does not have a closing tag called?

- A. Tag
- B. Empty element
- C. Closed element

10. Which of the following is an example of an empty element?

- A. `< img / >`
- B. `< img > < / img >`
- C. `< / img>`

11. What should values always be enclosed in?

- A. Quotation marks
- B. Commas
- C. Parenthesis

12. Where do all items for the same website need to be saved?

- A. In the same folder
- B. Where ever is fine
- C. In different folders

13. What does `< a href = "http : / / www . google . com" t
itle = "Link to Google " target = "_ blank " > Goog
le < / a > do?`

- A. Adds a link to google on the page
- B. Adds a search engine to the page
- C. Nothing

14. What is always a welcome page, and explains the purpose or topic of the site?

- A. Page 4
- B. Homepage
- C. Table of contents

15. What does View Source do?

- A. Nothing
- B. Brings up a note pad with the HTML code already used for the site.
- C. Opens a new website.



Answers key

1.A

8.C

2.A

9.B

3.B

10. A

4.A

11.A

5.A

12.A

6.B

13.A

7.B

14.B

15.B